

JTC Group

Target AI Architecture

Skill-driven AI platform — two reference deployment architectures on Azure

Open-source & portable · vs · Microsoft-managed

Reference architecture · v2.0 · Draft for review

The baseline platform (both options share this)

Both architectures run the same skill-driven AI platform — only the managed services underneath differ.

- Skill-driven: domain skill packs (single family office lead, tax, project management) loaded on demand — new domains ship as skills, not apps.
- Tools & data reached through MCP servers — one open, model-agnostic interface, reusable across every skill.
- LangGraph orchestrator routes each request to the right skill + agents.
- LLM via an OpenAI-compatible layer — the provider is a config switch, not a code change (Anthropic Claude default).
- Containerised app (Docker) with per-team isolation, RBAC, invites, chat logging, 👍/👎 feedback and LLM-judge evals built in.

Skill library — modular domain skill packs

The platform is skill-driven: each capability is a versioned skill pack (instructions + tools + MCP bindings) the orchestrator loads on demand. New domains ship as new skill packs — not new applications.

Single family office lead skills	Lead sourcing & qualification, relationship intelligence, prospect research, suitability & mandate drafting, onboarding / KYC
Tax skills	Form finding, obligation determination, FATCA/CRS readiness, W-8 preparation, regulatory-change monitoring with citations
Project management skills	Workstream & task planning, deadline / milestone tracking, status & RAID reporting, resourcing
Skill anatomy	Each pack = a SKILL.md (instructions) + tools + MCP bindings — hot-swappable, versioned and permission-scoped per team

MCP servers — standard tool & data access

Agents reach tools and data through Model Context Protocol (MCP) servers — a standard, pluggable interface any MCP-capable model (Anthropic Claude natively) can call. Skills bind to the servers they need.

Portfolio / entity MCP	Entities, obligations, filings, AEOI readiness
Document & RAG MCP	Tax-law corpus, retrieval, citations
CRM / relationship MCP	Contacts, mandates, pipeline — for the family-office lead skills
Filings & authority MCP	Form catalogues, e-file portals, deadlines
Office MCP	Email, calendar & documents (Microsoft 365 / Google)
Market & reference MCP	External market and reference data

Option 1 — Open-source & portable

RECOMMENDED

Managed Azure PaaS for the plumbing, open-source for the brain — no lock-in, fully pluggable LLMs with Anthropic Claude as default.

Azure Container Apps	Runs the FastHTML app (same Docker image)
Azure Blob Storage	Form PDFs & scraped source documents
Azure Database for PostgreSQL	Entities, obligations, chat logs, feedback & evals · pgvector for embeddings · optional Apache AGE for the citation graph
Azure AI Foundry (model catalog)	Pluggable LLMs — Anthropic Claude default (Opus 4.8 reasoning / Sonnet 4.6 cost); swap to OpenAI, Llama, Mistral with no code change
LangGraph + skill packs	Agent orchestrator loads the SFO-lead / tax / project-management skill packs on demand
MCP servers (containers)	Standard tool & data access — portfolio, docs, CRM, filings, office; reusable across every skill
Langfuse	Open-source LLM observability: traces, evals, cost, user feedback (self-host or Langfuse Cloud)
Key Vault + Entra ID	Secrets & single sign-on

Option 1 — how it flows

- User → Container Apps (FastHTML) over HTTPS with Entra SSO.
- LangGraph routes each request to specialist agents; every step is traced to Langfuse.
- LLM calls hit Azure AI Foundry via the OpenAI-compatible endpoint — Anthropic Claude by default, swappable per-agent.
- Retrieval runs over PostgreSQL (pgvector hybrid) + PDFs in Blob.
- 👍/👎 feedback and LLM-judge scores flow to Langfuse and the in-app analytics for a full quality loop.

Option 1 — pros & cons

Pros

- ✓ No lock-in — portable across Azure, other clouds, or on-prem.
- ✓ Truly pluggable LLMs: Anthropic default, swap freely; no single-vendor model risk.
- ✓ Reuses the existing codebase (LangGraph, pluggable storage) — minimal rework.
- ✓ Open-source observability you own (Langfuse) — full trace/eval/cost data.
- ✓ Transparent usage-based cost; scale each component independently.

Cons

- ✗ More moving parts to operate (Container Apps, Postgres, Langfuse).
- ✗ You own upgrades, scaling & backups — eased by managed PaaS.
- ✗ Graph queries need pgvector/Apache AGE, or a managed Neo4j AuraDB add-on.
- ✗ Langfuse is another service to run (or pay for Langfuse Cloud).

Option 2 — Microsoft-managed MANAGED

Foundry Agent Service + Microsoft Fabric — least ops, deepest Microsoft integration, at the cost of portability.

Azure AI Foundry Agent Service	Managed agents, built-in tool-calling, threads & multi-agent orchestration (replaces custom LangGraph)
Skills as connected agents	SFO-lead / tax / project-management skill packs map to Foundry connected agents & tool sets
MCP tools in Foundry	Foundry Agent Service consumes the same MCP servers as managed tools
Azure AI Search	Managed vector + hybrid RAG over the corpus
Microsoft Fabric (OneLake)	Unified data lake for tax docs · Lakehouse / Warehouse · Data Factory ingestion pipelines
Power BI	Firmwide compliance & filing dashboards on Fabric data
Azure OpenAI (GPT)	Default models; Anthropic Claude also available in the Foundry catalog
Azure Monitor + Foundry evaluations	Built-in tracing & evaluation
Microsoft Purview	Governance, data lineage & DLP

Option 2 — pros & cons

Pros

- ✓ Fully managed — least ops; Microsoft runs the agents, RAG and scaling.
- ✓ Deep Microsoft integration: Entra, Purview governance, Fabric, Power BI.
- ✓ Enterprise SLA & support — strong fit for a Microsoft-shop like JTC.
- ✓ Built-in agent orchestration, managed RAG and evaluations out of the box.
- ✓ Fabric unifies data + BI for firmwide reporting.

Cons

- ✗ Vendor lock-in — Azure/Foundry/Fabric-specific; low portability.
- ✗ Rework: migrate off LangGraph/Neo4j to Foundry Agents + Fabric.
- ✗ Model choice steered to OpenAI; less control over orchestration internals.
- ✗ Fabric capacity (F-SKUs) + Search + Foundry can be costly & hard to predict.
- ✗ Managed-agent surface is newer and still maturing.

Side by side

Dimension	Option 1 · Open-source	Option 2 · MS-managed
Compute	Container Apps (Docker)	Foundry Agent Service (managed)
Orchestration	LangGraph (code)	Foundry Agents (managed)
Skills	Portable skill packs (SKILL.md + tools)	Foundry connected agents / tool sets
Tool access	MCP servers (open standard, reusable)	MCP tools within Foundry
LLMs	Foundry catalog — Anthropic default, fully swappable	Azure OpenAI default; Claude via catalog
Data	PostgreSQL (+pgvector/AGE) + Blob	Fabric OneLake + AI Search
Observability	Langfuse (open-source)	Azure Monitor + Foundry evals
Analytics / BI	In-app + Langfuse	Power BI on Fabric
Lock-in	Low — portable	High — Azure-native
Ops burden	Higher	Lower
Rework	Minimal (reuses code)	Significant
Best for	Portability, model choice, cost control	MS-shop, managed ops, firmwide BI

Recommendation

- Default to Option 1 (open-source & portable): keeps Anthropic Claude as the primary model with freedom to swap, reuses the current LangGraph app, and avoids lock-in. Langfuse adds production-grade observability on top of the in-app analytics.
- Choose Option 2 when JTC wants a fully-managed, Microsoft-native stack with Fabric / Power BI for firmwide reporting and accepts Azure lock-in plus the migration effort.
- Hybrid path: start on Option 1 (fast, low-risk, portable), then adopt Fabric / Power BI for analytics later if firmwide BI becomes a priority — the pluggable storage and OpenAI-compatible LLM layer make that incremental, not a rebuild.
- The domain IP — the family-office lead, tax and project-management skill packs plus the MCP servers — is portable across both options, so the investment in skills carries over regardless of the deployment choice.